

Esercitazione: Costruisci un'Applicazione Spring Boot per Monitorare il Budget Personale

Obiettivi dell'esercitazione

In questa esercitazione svilupperai un'applicazione web utilizzando **Java 21** e **Spring Boot** seguendo l'architettura MVC. L'applicazione permetterà agli utenti di monitorare il proprio budget personale registrando entrate e uscite, categorizzando le spese, caricando ricevute e consultando statistiche finanziarie.

Durante lo sviluppo utilizzerai:

- Spring Boot
- Spring MVC
- Spring Data JPA (Hibernate)
- Spring Security
- MySQL
- REST API
- Thymeleaf
- Maven
- Validazione dei dati
- Upload di file
- Gestione delle eccezioni

Funzionalità richieste

1. Registrazione e Login

Gli utenti devono poter:

- Registrarsi.
- Effettuare il login.
- Effettuare il logout.

Le password devono essere memorizzate in modo sicuro utilizzando **BCryptPasswordEncoder** fornito da Spring Security.

2. Autenticazione e Sicurezza

Proteggi l'applicazione utilizzando **Spring Security**.

Solo gli utenti autenticati devono poter:

- visualizzare le proprie transazioni;
- aggiungere nuove entrate e uscite;
- modificare il proprio profilo;

- accedere alla dashboard.

Le pagine pubbliche saranno solamente:

- Login
 - Registrazione
-

3. Gestione delle Transazioni

L'utente deve poter:

- inserire una nuova transazione;
- modificare una transazione;
- eliminare una transazione;
- visualizzare l'elenco delle proprie transazioni.

Ogni transazione deve contenere:

- descrizione;
- data;
- importo;
- tipo (Entrata/Uscita);
- categoria;
- ricevuta (facoltativa).

Categorie di esempio:

- Affitto
 - Stipendio
 - Spese Generali
 - Trasporti
 - Tempo Libero
 - Altro
-

4. Upload delle Ricevute

L'utente può caricare:

- immagini JPG
- PNG
- PDF

Il file deve essere salvato in una cartella del server mentre nel database verrà memorizzato solamente il percorso del file.

5. Dashboard Finanziaria

Realizzare una dashboard che mostri:

- saldo totale;
- totale delle entrate;
- totale delle uscite;
- spese suddivise per categoria;
- bilancio mensile.

(Opzionale: visualizzare grafici utilizzando Chart.js.)

6. Gestione del Profilo

L'utente deve poter modificare:

- nome;
- email;
- obiettivi finanziari;
- password.

7. REST API

Realizzare una REST API per la gestione delle transazioni.

Endpoint richiesti:

Metodo	Endpoint	Descrizione
GET	/api/transactions	Elenco transazioni
GET	/api/transactions/{id}	Dettaglio transazione
POST	/api/transactions	Inserimento
PUT	/api/transactions/{id}	Modifica
DELETE	/api/transactions/{id}	Eliminazione

Le API devono utilizzare il formato **JSON**.

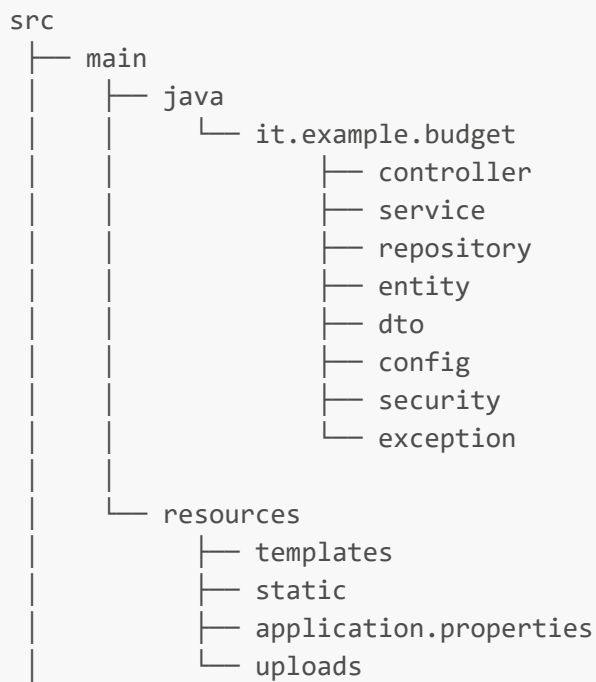
Requisiti Tecnici

L'applicazione deve utilizzare:

- Java 21
- Spring Boot
- Spring MVC
- Spring Data JPA
- Hibernate
- Spring Security
- Thymeleaf

- Maven
- MySQL
- Bean Validation
- Lombok (opzionale)

Architettura del Progetto



Entity

User

Campi:

- id
- name
- email
- password
- financialGoal
- createdAt

Relazione:

Un utente può possedere molte transazioni.

```
User
|
|-----< Transaction
```

Transaction

Campi:

- id
- description
- date
- amount
- type
- category
- receipt
- createdAt

Relazione:

Molte transazioni appartengono ad un solo utente.

Repository

Creare i repository:

- UserRepository
- TransactionRepository

estendendo **JpaRepository**.

Service

Creare i servizi:

UserService

Responsabilità:

- registrazione
 - login
 - modifica profilo
 - cambio password
-

TransactionService

Responsabilità:

- CRUD transazioni
 - calcolo statistiche
 - ricerca per categoria
 - ricerca per data
-

Controller

AuthController

Gestisce:

- login
 - logout
 - registrazione
-

UserController

Gestisce:

- profilo utente
 - modifica dati personali
-

TransactionController

Gestisce:

- elenco transazioni
 - inserimento
 - modifica
 - eliminazione
 - dashboard
-

TransactionRestController

Espone la REST API.

View (Thymeleaf)

Realizzare le seguenti pagine:

- login.html
- register.html
- dashboard.html
- transactions.html
- transaction-form.html
- profile.html

Database MySQL

Tabella users

```
CREATE TABLE users (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  financial_goal VARCHAR(255),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Tabella transactions

```
CREATE TABLE transactions (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  user_id BIGINT NOT NULL,  
  description VARCHAR(255),  
  date DATE NOT NULL,  
  amount DECIMAL(10,2),  
  type ENUM('ENTRATA', 'USCITA'),  
  category VARCHAR(100),  
  receipt VARCHAR(255),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  FOREIGN KEY(user_id)  
  REFERENCES users(id)  
  ON DELETE CASCADE  
);
```

Dati Fake

Users

```
INSERT INTO users(name,email,password)
VALUES
('Giovanni Rossi','giovanni@example.com','$2a$10$abc123...'),
('Maria Bianchi','maria@example.com','$2a$10$def456...'),
('Luca Verdi','luca@example.com','$2a$10$ghi789...');
```

Transactions

```
INSERT INTO transactions(user_id,description,date,amount,type,category,receipt)
VALUES
(1,'Affitto Settembre','2025-09-01',850.00,'USCITA','Affitto','uploads/affitto.jpg'),
(2,'Stipendio','2025-09-05',1500.00,'ENTRATA','Stipendio',NULL),
(3,'Supermercato','2025-09-10',80.50,'USCITA','Spese Generali','uploads/scontrino.jpg');
```

Passaggi dell'Esercitazione

Parte 1

- Creazione del progetto Spring Boot.
- Configurazione Maven.
- Collegamento a MySQL.

Parte 2

Creazione delle Entity:

- User
- Transaction

con le relative annotazioni JPA.

Parte 3

Creazione dei Repository.

Parte 4

Realizzazione dei Service contenenti la logica di business.

Parte 5

Configurazione di Spring Security.

- Login
 - Logout
 - BCrypt
 - Protezione delle pagine
-

Parte 6

Realizzazione delle pagine Thymeleaf.

Parte 7

Implementazione del caricamento dei file.

Parte 8

Realizzazione della Dashboard con statistiche finanziarie.

Parte 9

Implementazione della REST API.

Parte 10

Gestione delle eccezioni con:

- @ControllerAdvice
 - ExceptionHandler
-

Obiettivi Formativi

Al termine dell'esercitazione lo studente sarà in grado di:

- sviluppare un'applicazione completa con Spring Boot;
- utilizzare Spring MVC e Thymeleaf;
- implementare autenticazione con Spring Security;
- progettare Entity e Repository con Spring Data JPA;
- creare servizi separando la logica di business;
- realizzare REST API secondo lo stile RESTful;
- utilizzare MySQL con Hibernate;
- implementare upload di file;

- applicare il pattern MVC;
 - utilizzare la validazione dei dati e la gestione centralizzata delle eccezioni.
-

Conclusione

Al termine dell'esercitazione avrai realizzato un'applicazione completa per la gestione del budget personale basata su Spring Boot, comprendente autenticazione sicura, gestione utenti, CRUD delle transazioni, dashboard finanziaria, caricamento delle ricevute e una REST API utilizzabile anche da applicazioni esterne.